

GitHub:

<https://github.com/janaashraf888/rowadmasrtasks.git>

LinkedIn posts:

1)

https://www.linkedin.com/posts/janaashraf88_softwarearchitecture-designpatterns-mvc-activity-7439358106993573888-22uB?utm_source=share&utm_medium=member_desktop&rcm=ACoAAE8YvOUBdlbw_Me3Yv_nmlqA8meTZp5UEhc

2) https://www.linkedin.com/posts/janaashraf88_dotnet-mvc-webdevelopment-activity-7439357049433636864-DAeT?utm_source=share&utm_medium=member_desktop&rcm=ACoAAE8YvOUBdlbw_Me3Yv_nmlqA8meTZp5UEhc

Self Study:

1. Builder Design Pattern

The **Builder Pattern** is a creational design pattern used to construct complex objects step-by-step. It allows you to produce different types and representations of an object using the same construction code.

- **When to use:** When an object requires many optional parameters or has a complex initialization process (the "Telescoping Constructor" problem).
- **Key Benefit:** It improves code readability and separates the representation of an object from its construction logic.

2. Dependency Injection: Chaining Parameter Constructors

In the context of configuring connections (like a Database Context), **Dependency Injection (DI)** is the technique of providing the objects that a class needs instead of having the class construct them itself.

- **Chaining Constructors:** This involves one constructor calling another constructor within the same class (using the `this` keyword) or a base class (using the `base` keyword).
- **Workflow:** Usually, a parameterless constructor is chained to a more complex one, ensuring that default configurations or connection strings are injected correctly by the DI container during application startup.

3. Soft Delete

A **Soft Delete** is a data management strategy where a record is not physically removed from the database. Instead, a flag (usually a boolean `IsDeleted` or a timestamp `DeletedAt`) is updated to mark the record as "hidden."

- **Pros:** Allows for easy data recovery, maintains referential integrity, and provides an audit trail.
- **Cons:** Increases database size over time and requires all "Select" queries to filter out deleted rows (e.g., `WHERE IsDeleted = false`).

4. Restrict vs. NoAction (SQL Constraints)

These refer to Referential Integrity actions when a parent record is deleted or updated.

- **Restrict:** Prevents the deletion of a parent record if any child records exist. It is a strict "hard stop."

- **NoAction:** Similar to Restrict, it prevents the deletion, but the check is deferred until the end of the transaction. In many modern databases (like SQL Server), NoAction is the default behavior and effectively behaves like Restrict.

5. Data Seeding

Data Seeding is the process of populating a database with an initial set of data.

- **Purpose:** It is used to provide essential lookup data (like a list of countries or roles), configuration settings, or "dummy data" for testing environments.
- **Implementation:** Usually handled within the OnModelCreating method in Entity Framework or through dedicated migration scripts.

6. Lazy Loading vs. Eager Loading

These are two primary strategies for fetching related data from a database.

Feature	Lazy Loading	Eager Loading
Definition	Related data is loaded only when it is specifically accessed in the code.	Related data is loaded as part of the initial query.
Performance	Can lead to the "N+1 Problem" (too many database calls).	Can result in large, complex queries with many Joins.

Feature	Lazy Loading	Eager Loading
Best For	When you aren't sure if you will need the related data.	When you know for certain the related data is required immediately.
Syntax (EF)	Uses virtual properties.	Uses the .Include() method.